MANUAL TÉCNICO

Repositorio Documental y Gestión de Trabajos Especiales de Grado (TEG)

VERSIÓN	FECHA	INSTITUCIÓN	CLASIFICACIÓN
2.0.0 (Modular)	Octubre 2026	IUTEPAL	Técnica Interna

📌 1. Ficha de Traspaso y Visión General del Proyecto

Este manual garantiza una transferencia tecnológica limpia y exhaustiva para el analista o ingeniero responsable del mantenimiento, evolución y despliegue del repositorio institucional del **IUTEPAL**. Aquí se detalla la radiografía completa de la plataforma.

1.1. PROPÓSITO DEL SISTEMA

El sistema centraliza, custodia, indexa y difunde los **Trabajos Especiales de Grado (TEG)**, Hojas de Resumen y Presentaciones Académicas de las carreras de Informática, Producción Industrial, Electrónica, Contabilidad, Administración y Preescolar, ofreciendo auditoría de accesos en tiempo real.

1.2. HITOS DE LA VERSIÓN 2.0.0 (REFACTORIZACIÓN)

En versiones anteriores, el sistema dependía de servicios en la nube (MySQL en Clever Cloud y archivos en Cloudinary). La versión 2.0.0 introduce una **arquitectura en capas desacoplada** con cuatro avances clave:

01 Autonomía 100% Offline
Capacidad de ejecutarse en red local o Intranet institucional sin requerir conexión a Internet.

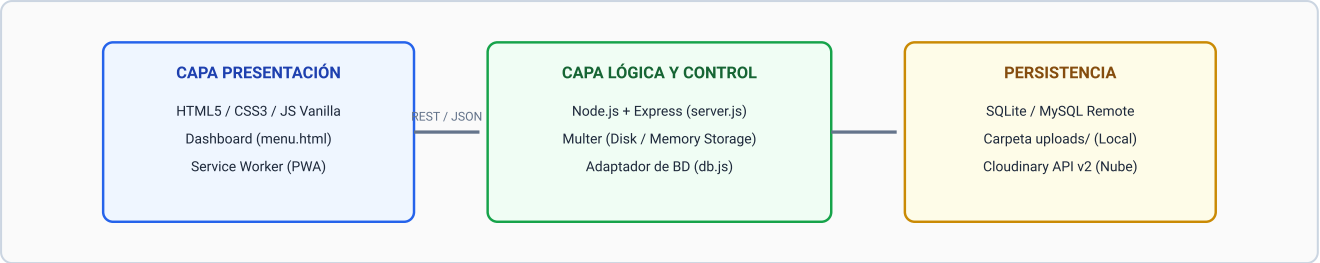
02 Motor de Datos Híbrido
Soporte unificado en `db.js` para SQLite embebido y MySQL/MariaDB con pool de conexiones.

03 Almacenamiento Local
Subida y servicio directo de archivos vía Multer en la carpeta `uploads/` mediante rutas estáticas.

04 Reversibilidad Multi-Nube
Conmutación en segundos entre entornos local y nube mediante variables de entorno y scripts CLI.

🏗️ 2. ARQUITECTURA DE SOFTWARE Y TOPOLOGÍA

El sistema está estructurado bajo una arquitectura cliente-servidor desacoplada en tres capas principales: Presentación, Control/Lógica y Persistencia de Datos.



2.1. CICLO DE VIDA DE UNA PETICIÓN (REQUEST LIFECYCLE)

1. **Petición del Cliente:** El frontend emite una solicitud HTTP (ej: `POST /upload` con `FormData`).
2. **Procesamiento Middleware:** Express parsea headers y ejecuta Multer.
3. **Decisión de Almacenamiento:** Si `STORAGE_TYPE=local`, Multer guarda en `uploads/` con nombre saneado y sufijo de tiempo, retornando `/uploads/nombre.pdf`. Si es `cloudinary`, transmite el buffer en memoria hacia Cloudinary API.
4. **Persistencia:** Se invocan metadatos a `POST /addTrabajo`. El servidor llama a `db.execute()` enviando la sentencia de inserción.
5. **Agnosticismo del Motor:** El archivo `db.js` traduce la consulta tanto para SQLite como para MySQL, retornando el formato estandarizado `[rows, meta]`.

⚙️ 3. MATRIZ EXHAUSTIVA DE VARIABLES DE ENTORNO (.ENV)

Toda la configuración operativa se concentra en el archivo `.env`. Ningún cambio de infraestructura requiere modificar el código fuente.

Variable	Tipo	Permitidos	Propósito Técnico
`PORT`	Numérico	3000, 8080	Puerto TCP donde escucha el servidor Express.
`STORAGE_TYPE`	String	local | cloudinary	Ubicación de nuevas cargas (disco local o nube).
`UPLOAD_DIR`	String	uploads	Directorio físico para almacenamiento local.
`DB_CLIENT`	String	sqlite | mysql	Selecciona el motor de base de datos a instanciar.
`SQLITE_DB_PATH`	Ruta	./data/database.sqlite	Ruta física del archivo SQLite embebido.
`DB_HOST / DB_PORT`	Host / Num	localhost / 3306	Servidor y puerto de conexión MySQL.
`DB_USER / DB_PASS`	String	root / (vacío)	Credenciales del servidor MySQL local o remoto.
`MYSQL_ADDON_URI`	URI	mysql://user:pass...	Conexión PaaS (Clever Cloud/Railway). Anula DB_HOST.
`CLOUDINARY_*`	String/Hash	NAME, KEY, SECRET	Credenciales de la API activa de Cloudinary.

3.1. Perfiles de Configuración Destacados

```
# PERFIL 1: Entorno 100% Local y Offline (Por Defecto)
```

```
PORT=3000 STORAGE_TYPE=local UPLOAD_DIR=uploads DB_CLIENT=sqlite
SQLITE_DB_PATH=./data/database.sqlite # PERFIL 2: Entorno Local con MySQL / XAMPP
PORT=3000 STORAGE_TYPE=local DB_CLIENT=mysql DB_HOST=localhost DB_USER=root
DB_NAME=document_upload
```

🔄 4. MANUAL DE OPERACIONES Y MIGRACIONES CLI

El repositorio cuenta con scripts automatizados en la carpeta `scripts/` para gestionar el ciclo de vida de los datos.

4.1. Siembra de Datos Iniciales (Seed)

Inicializa la base de datos (SQLite o MySQL) cargando los 5 usuarios fundacionales, 25 trabajos y 15 registros de auditoría de forma idempotente (`INSERT OR REPLACE`):

```
npm run seed
```

4.2. Migrar de Cloudinary a Almacenamiento Local

Descarga físicamente todos los documentos desde URLs de Cloudinary a la carpeta `uploads/` y actualiza la base de datos a rutas locales:

```
npm run migrate # O: node scripts/migrate.js --target=local --force
```

4.3. Retorno a la Nube (Migrar de Local a Cloudinary)

1. Ajustar en `.env` : `STORAGE_TYPE=cloudinary` y las claves de API.
2. Subir masivamente los archivos locales a la nube e implementar URLs puras:

```
npm run migrate:cloudinary
```

3. Conectar la base de datos a MySQL remoto mediante `MYSQL_ADDON_URI` o credenciales directas.

4.4. Migración entre Cuentas de Cloudinary

Configurando `NEW_CLOUDINARY_*` en el archivo `.env` , el comando `npm run migrate:cloudinary` migra recursos de la cuenta antigua a la nueva automáticamente.

🗄️ 5. MODELO RELACIONAL Y DICCIONARIO DE DATOS

5.1. Tabla: usuario

Campo	Tipo	Nulo	Descripción
`id`	INT (PK)	NO	ID autoincremental. ID 0 reservado para Admin maestro.
`email`	VARCHAR(100)	NO	Correo para inicio de sesión de estudiantes y visitantes.
`esIutepalista`	TINYINT	SI	1 = Estudiante IUTEPAL (acceso a Resúmenes), 0 = Visitante.

5.2. Tabla: trabajos

Campo	Tipo	Nulo	Descripción
`id`	VARCHAR(50)	NO	Clave primaria. Formato alfanumérico 't' + timestamp.
`nombre`	TEXT	SI	Título completo de la tesis o documento académico.
`carrera`	VARCHAR(50)	SI	INFORMATICA, PRODUCCION, ELECTRONICA, etc.
`tipo`	VARCHAR(20)	SI	trabajo, resumen, presentacion, video.
`pdf_url`	TEXT	SI	Puntero al recurso (/uploads/file.pdf o URL Cloudinary).

5.3. Tabla: logs

Campo	Tipo	Nulo	Descripción
`id`	INT (PK)	NO	Identificador autoincremental de auditoría.
`usuario_id`	INT (FK)	SI	Referencia a usuario.id (ON DELETE CASCADE).
`action`	VARCHAR(50)	NO	Tipo de evento: access, view, download.
`doc_id`	VARCHAR(50)	SI	Referencia a trabajos.id (ON DELETE SET NULL).

🚨 6. PROTOCOLO DE RESOLUCIÓN DE INCIDENCIAS (RUNBOOK)

Procedimientos paso a paso para resolver los fallos operativos más frecuentes en entornos de producción y pruebas:

Incidencia 1: Error EADDRINUSE :::3000

Causa: Un proceso previo de Node.js o un servicio externo está ocupando el puerto 3000.
Solución: En Windows PowerShell ejecutar `netstat -ano | findstr :3000` y luego finalizar el proceso con `taskkill /PID <PID> /F`. Alternativamente, modificar temporalmente `PORT=3001` en el archivo `.env`.

Incidencia 2: Error ECONNREFUSED en MySQL

Causa: El servicio MySQL/MariaDB (XAMPP/WAMP) está detenido.
Solución: Iniciar el servicio desde el panel de control. Si se desea operar sin MySQL, cambiar en `.env` la variable `DB_CLIENT=sqlite` para activar instantáneamente la base de datos local embebida.

Incidencia 3: Error EBUSY: resource busy or locked

Causa: Un visor PDF externo (ej. Adobe Reader) tiene bloqueado un archivo en `uploads/` o el archivo `database.sqlite` está abierto en un cliente GUI.
Solución: Cerrar los visores o editores locales. SQLite en modo WAL reduce bloqueos, pero no evita el bloqueo exclusivo de Windows sobre archivos abiertos.

Incidencia 4: Reporte de actividad no carga

Causa: Registros con campos nulos procedentes de migraciones incompletas.
Solución: El frontend actual valida arreglos con `Array.isArray()` y asigna fallback para nombres vacíos (`log.nombre || 'Usuario desconocido'`).

🔒 7. RECOMENDACIONES DE SEGURIDAD Y EVOLUCIÓN (V3.0)

Propuestas de mejora estructural recomendadas para la próxima versión del sistema:

01. Hashing de Contraseñas
Implementar `bcrypt` o `argon2` en los endpoints de registro y login para eliminar el almacenamiento en texto plano.

02. Autenticación con JWT
Sustituir el control en `localStorage` por tokens JWT firmados enviados en Cookies HttpOnly y validados mediante middleware.

03. Validación MIME de Subidas
Configurar `fileFilter` en Multer verificando cabeceras de bytes (Magic Numbers) para prevenir la subida de archivos maliciosos.

04. Respaldos Programados
Integrar `node-cron` para realizar copias de seguridad automáticas diarias de la carpeta `uploads/` y de la base de datos.

🚀 COMANDOS RÁPIDOS DE REFERENCIA

Comandos principales listados en el `README.md` del proyecto:**Inicio Rápido (Entorno Local)**

1. REQUISITOS

- Node.js v18 o superior.
- NPM v9 o superior.

2. INSTALACIÓN Y EJECUCIÓN

1. Clonar el repositorio e ingresar a la carpeta: `cd CODIGOCOMPARTIDOIUTEPAL`
2. Instalar dependencias: `npm install`
3. Iniciar el servidor local: `npm start`
4. Abrir en el navegador: `http://localhost:3000`

Credenciales Predetermadas

Rol	Usuario / Correo	Contraseña	Permisos y Alcance
Administrador	`admin`	`123`	Control total (subir, editar, eliminar trabajos, cambiar contraseñas y ver reportes).
Estudiante IUTEPAL	`test@correo.com` *(o `ejemplo@gmail.com`)*	`123456` *(o `123`)*	Acceso preferencial a Hojas de Resumen y visualización de documentos.
Visitante	`otro@gmail.com`	`123`	Consulta y descarga de trabajos de grado